GraphRAG System

Technical Reference & Design Document

Xerox WorkCentre 3025 Manual — Knowledge-Graph Augmented Retrieval

System	GraphRAG — Graph + Vector Hybrid Retrieval
Document	Xerox WorkCentre 3025 User Manual
Pipelines	Offline Indexing + Online Query
Stack	Python · FAISS · sentence-transformers · CSV
Status	Working — 3 fixes recommended

Contents

1. System Overview

This document describes a Graph-augmented Retrieval system (GraphRAG) built to answer natural-language questions against the Xerox WorkCentre 3025 user manual. The system combines two complementary retrieval strategies — dense vector search and structured knowledge-graph traversal — to surface high-confidence, page-cited context.

1.1 Architecture at a glance

Component	Description
PDF extraction	PyMuPDF converts the printer manual into a JSON list of {page, text} records.
Text chunker	Sliding-window splits produce overlapping 1,200-char chunks (overlap 150 chars) to prevent context loss at boundaries.
Regex NER	8 entity types (MODEL, FEATURE, MENU, PROTOCOL, COMPONENT, PROBLEM, ACTION, SETTING) are extracted via compiled regex patterns.
Co-occurrence graph	Entity pairs that co-occur within a chunk form directed triples with one of 7 inferred relation types. Triples are written to graph_triples.csv.
Vector index	all-MiniLM-L6-v2 encodes every chunk into a 384-dim L2-normalised embedding stored in a FAISS IndexFlatIP.
Query engine	At query time, both the vector index and triple store are searched. Results are printed to stdout for analyst review.

1.2 Two-pipeline design

The system operates in two phases:

- **Pipeline A (offline):** runs once when the corpus changes. Produces 5 persistent artifacts.
- **Pipeline B (online):** runs on every user question. Reads the artifacts, scores them, and prints results.

2. Pipeline A — Offline Indexing

Trigger: manual run when document corpus changes | Script: manual_rag.py

2.1 Step 1 — load_pages()

Reads the JSON output from PyMuPDF and filters out any records with null page or text fields. The result is capped at MAX_PAGES = 100.

```
INPUT : manual_unstructured_pymupdf.json
        [{"page": 1, "text": "..."}, ...]

data  = json.loads(INPUT_JSON.read_text())
pages = [row for row in data
         if row.get('page') and row.get('text')]
pages = pages[:MAX_PAGES]   # hard cap — emits warning if truncated

OUTPUT: list[{page:int, text:str}]  len <= 100
COMPLEXITY: O(n)
```

2.2 Step 2 — build_chunks()

Applies a sliding-window algorithm to each page's text. Whitespace is collapsed before splitting. Chunks shorter than 1 character are discarded.

```
CONFIG: CHUNK_SIZE = 1200 chars
        CHUNK_OVERLAP = 150 chars
        stride = CHUNK_SIZE - CHUNK_OVERLAP = 1050 chars

FOR page IN pages:
    text  = re.sub(r'\s+', ' ', page.text).strip()
    start = 0
    WHILE start < len(text):
        end   = start + CHUNK_SIZE
        emit  {chunk_id, page, text[start:end]}
        IF end >= len(text): BREAK
        start = end - CHUNK_OVERLAP

WRITES: chunks.jsonl   (one JSON record per line)
COMPLEXITY: O(P * T/stride)  where P=pages, T=avg text len
```

2.3 Step 3 — extract_entities()

Runs 43 pre-compiled regex patterns across 8 entity type categories against every chunk. Each match is normalised and recorded with its count, source pages, and source chunk IDs.

Entity type catalogue

Type	Example match	Regex pattern (abbreviated)

MODEL	WorkCentre 3025NI	`WorkCentre\s+3025(BI|NI)`
FEATURE	Wi-Fi Direct	`Wi-?Fi Direct|AirPrint|Fax|Scan`
MENU	Machine Status	`Machine Status|System Setup|Wi-?Fi Settings`
PROTOCOL	TCP/IP	`TCP/IP|IPv4|IPv6|DHCP|DNS|SNMPv3?`
COMPONENT	Control Panel	`Control Panel|Paper Tray|USB Port`
PROBLEM	Paper Jam	`Paper Jam|Error Messages?|Poor Print Quality`
ACTION	Configure	`Press|Select|Enable|Configure|Restart`
SETTING	admin	`Administrator Password|SSID|WPA Key|1111`

```
compiled = {label: [re.compile(p, re.I) for p in patterns]}

FOR chunk IN chunks:
    FOR (label, regex_list) IN compiled.items():
        FOR regex IN regex_list:
            FOR match IN regex.finditer(chunk.text):
                name = normalize(match.group(0))
                entities[name].count++
                entities[name].pages.add(chunk.page)
                entities[name].chunks.add(chunk.chunk_id)

normalize(value): re.sub(r'\s+', ' ', value).strip()
  *** BUG: does not .lower() — 'Network' != 'network' ***

WRITES: entities.json
COMPLEXITY: O(C * T * P_total)
```

2.4 Step 4 — build_graph()

For every chunk, all unique entity pairs are combined (upper triangle only) and a relation type is inferred purely from the source and target entity types. A deduplication key of (source, relation, target, page) prevents exact-duplicate triples.

```
FOR chunk IN chunks:
    unique = deduplicate(chunk_entities[chunk_id])
    FOR i, src IN enumerate(unique):
        FOR tgt IN unique[i+1:]:
            relation = infer_relation(src.type, tgt.type)
            key = (src.name, relation, tgt.name, chunk.page)
            IF key NOT IN seen: emit triple

infer_relation(src_type, tgt_type):
    MODEL   + FEATURE   -> HAS_FEATURE
    FEATURE + MENU      -> CONFIGURED_VIA
    FEATURE + PROTOCOL  -> USES_PROTOCOL
    PROBLEM + ACTION    -> RESOLVED_OR_HANDLED_BY
    PROBLEM + COMPONENT -> INVOLVES_COMPONENT
    MENU    + SETTING   -> HAS_SETTING
    *       + *         -> RELATED_TO   *** majority fallback ***
```

```
WRITES: graph_triples.csv
SCHEMA: source, relation, target, page, chunk_id, evidence[:300]
COMPLEXITY: O(C * E^2)  where E = avg entities per chunk
```

2.5 Step 5 — build_vector_index()

All chunk texts are batch-encoded by the sentence transformer model. L2 normalisation converts raw embeddings to unit vectors so that inner product equals cosine similarity. The resulting index supports exact brute-force search.

```
model      = SentenceTransformer('all-MiniLM-L6-v2')
texts      = [c['text'] for c in chunks]
embeddings = model.encode(texts,
                normalize_embeddings=True,  # L2-norm
                show_progress_bar=True)
embeddings = np.array(embeddings).astype('float32')

index = faiss.IndexFlatIP(384)   # cosine via inner product
index.add(embeddings)

WRITES: manual.faiss          (binary)
        chunk_metadata.json   (list — position i == index id i)

CRITICAL: index position i maps 1-to-1 to chunks[i].
          Do not reorder chunks after writing the index.
```

2.6 Output artifacts

File	Format	Contents
chunks.jsonl	JSONL	One record per chunk: chunk_id, page, text
entities.json	JSON	Entity list + chunk_entities map: name, type, count, pages[], chunks[]
graph_triples.csv	CSV	source, relation, target, page, chunk_id, evidence[:300]
manual.faiss	Binary	IndexFlatIP, dim=384, n=chunk count, float32
chunk_metadata.json	JSON	Ordered list of chunk records. Position i = FAISS id i

3. Pipeline B — Online Query

Trigger: each user question | Script: test_rag.py | No external API

The query pipeline reads the five build-time artifacts and produces ranked retrieval results printed to stdout. No LLM or external API is involved.

3.1 Startup — load artifacts once

All heavy resources should be loaded at module level, not inside per-call functions. In the current code they are loaded inside vector_search() which means they reload on every question.

```
# Correct pattern — module level (not inside function):
_model   = SentenceTransformer('sentence-transformers/all-MiniLM-L6-v2')
_index   = faiss.read_index(str(OUTPUT_DIR / 'manual.faiss'))
_chunks  = json.loads((OUTPUT_DIR / 'chunk_metadata.json').read_text())
_triples = list(csv.DictReader(open(OUTPUT_DIR / 'graph_triples.csv')))
```

3.2 Step B1 — vector_search(question, top_k=5)

```
INPUT  : question string
OUTPUT : list[{score, chunk_id, page, text}]

q_emb = model.encode([question],
            normalize_embeddings=True).astype('float32')
# shape (1, 384) — unit vector — cosine search ready

scores, ids = index.search(q_emb, top_k)
# IndexFlatIP + L2-norm => cosine similarity
# ids[i] == -1  =>  FAISS sentinel, skip

FOR rank, idx IN enumerate(ids[0]):
    IF idx == -1: continue
    chunk = chunks[idx]          # O(1) list position lookup
    yield {score, chunk_id, page, text}
```

3.3 Step B2 — graph_search(question, vector_results, limit=20)

```
INPUT  : question, vector_results
OUTPUT : list[triple], sorted desc by score

# Tokenise (with stopword fix applied)
STOPWORDS = {'how','what','why','when','the','and',
             'for','are','can','you','do','is','to'}
terms = [t.lower() for t in re.findall(r'[A-Za-z0-9\-]+', question)
         if len(t) > 2 and t.lower() not in STOPWORDS]

hit_chunks = {r['chunk_id'] for r in vector_results}
```

```
    FOR triple IN triples:
        score = 0
        IF triple['chunk_id'] IN hit_chunks: score += 5
        blob = (source + relation + target + evidence).lower()
        FOR term IN terms:
            IF term IN blob: score += 1
        IF score > 0: keep(score, triple)

    sort desc, deduplicate by (source, relation, target)
    return top limit=20
```

3.4 Step B3 — test_question() output

The function prints vector results (score, page, chunk, text[:800]) followed by graph results (source, relation, target, page, chunk_id) to stdout. This is the final output of the existing code — no LLM call is made.

```
    def test_question(question):
        vector_results = vector_search(question)
        graph_results  = graph_search(question, vector_results)

        print('VECTOR RESULTS')
        for r in vector_results:
            print(f"Page: {r['page']} | Chunk: {r['chunk_id']}
                   | Score: {r['score']:.3f}")
            print(r['text'][:800])

        print('GRAPH RESULTS')
        for r in graph_results:
            print(f"{r['source']} --{r['relation']}--> {r['target']}
                   | page {r['page']} | chunk {r['chunk_id']}")
```

4. Traced Example — "How do I configure Wi-Fi?"

Real output from test run — annotated step by step

4.1 Question tokenisation

The question is parsed into search terms using a regex that extracts alphanumeric tokens. Terms with fewer than 3 characters are discarded.

Token	Kept?	Reason
`how`	**REMOVED (fix)**	Stopword — matches noise in evidence
`do`	**Dropped**	len < 3
`configure`	**KEPT**	Strong content signal
`wi-fi`	**KEPT**	Entity name — direct match

4.2 Vector search results

FAISS searches the index for the 5 chunks closest to the encoded question embedding. All returned chunks are highly relevant (cosine > 0.52).

Chunk ID	Page	Score	Key content
`chunk_99`	44	**0.600**	CentreWare Internet Services — admin/1111 → Properties → Network Settings → Wi-Fi → Easy/Advanced Wizard
`chunk_100`	44	0.593	Advanced Wireless Setup — Wireless Radio On, SSID list, Authentication, Encryption, Network Key
`chunk_91`	40	0.586	Wi-Fi Direct, Wi-Fi Signal, Wi-Fi Default options; Wizard vs Custom setup path
`chunk_90`	40	0.530	Machine Status → Network → enter 1111 → Wi-Fi menu
`chunk_124`	56	0.529	Enter Network Password → Next → Finish

`{chunk_90, chunk_91, chunk_99, chunk_100, chunk_124}` — these 5 IDs pass a +5 bonus to the graph scorer.

4.3 Graph search score breakdown

Each triple is scored by two signals: whether it originated from a vector-confirmed chunk (+5), and how many question terms appear in the concatenated (source + relation + target + evidence) string (+1 each).

```
Example — top-scoring triple:

triple  : Wi-Fi --CONFIGURED_VIA--> Machine Status
chunk_id: chunk_90   (in hit_chunks)
```

```
score = 0
+ 5   chunk_90 in hit_chunks           (vector alignment)
+ 1   'wi-fi' found in source blob
+ 1   'configure' found in evidence
──────────────────────────────────────
= 7   final score
```

4.4 Graph results — annotated

The actual 20 rows printed by the run, with bug classification:

Triple	Status	Issue
Wi-Fi --CONFIGURED_VIA--> Machine Status (p40)	VALID	—
Wi-Fi --CONFIGURED_VIA--> Network (p40)	VALID	—
Wi-Fi --CONFIGURED_VIA--> network (p40)	DUP	Case duplicate — normalize_entity() missing .lower()
Wi-Fi --CONFIGURED_VIA--> Wi-Fi Settings (p40)	VALID	—
Wi-Fi --CONFIGURED_VIA--> Wi-Fi Settings (p44)	DUP	Cross-page duplicate — missing (src,rel,tgt) dedup
Wi-Fi --CONFIGURED_VIA--> Network (p44)	DUP	Cross-page duplicate
Wi-Fi --CONFIGURED_VIA--> network (p44)	DUP	Case + page duplicate
Wi-Fi --CONFIGURED_VIA--> Network Settings (p44)	VALID	—
Wi-Fi --RELATED_TO--> Configure (p40)	WEAK	Fallback relation — no specific rule matched
Wi-Fi --RELATED_TO--> configure (p44)	DUP	Case duplicate of above
Print --CONFIGURED_VIA--> Wi-Fi Settings (p40)	NOISE	Co-occurrence artefact — Print and Wi-Fi Settings share chunk_90 but are unrelated
Wi-Fi Direct --CONFIGURED_VIA--> Machine Status	VALID	—
Wi-Fi Direct --CONFIGURED_VIA--> Network	VALID	—
Wi-Fi Direct --CONFIGURED_VIA--> network	DUP	Case duplicate
Wi-Fi Direct --CONFIGURED_VIA--> Wi-Fi Settings	VALID	—
CentreWare IS --CONFIGURED_VIA--> Wi-Fi Settings	VALID	—
Xerox WorkCentre --RELATED_TO--> Configure	WEAK	Fallback relation
Xerox WorkCentre --RELATED_TO--> configure	DUP	Case duplicate
Wi-Fi Settings --RELATED_TO--> Configure	WEAK	Fallback relation

Summary: 20 rows printed → 8 valid, 9 duplicates, 1 noise, 3 weak fallback. After fixes: 10 clean unique triples.

5. Bug Report & Recommended Fixes

> **All fixes apply to existing Python files only — no new dependencies**

5.1 Fix 1 — Case normalisation in entity extraction

File: manual_rag.py **|** **Function:** normalize_entity()

Impact: Eliminates 6 of 20 duplicate graph rows for the Wi-Fi query alone.

```
# BEFORE (current — buggy):
def normalize_entity(value):
    return re.sub(r'\s+', ' ', value).strip()

# AFTER (fixed):
def normalize_entity(value):
    return re.sub(r'\s+', ' ', value).strip().lower()

# Result: 'Network' and 'network' now resolve to the same entity.
# Re-run build pipeline after applying this change.
```

5.2 Fix 2 — Triple deduplication at query time

File: test_rag.py **|** **Function:** graph_search()

Impact: Reduces printed graph output from 20 rows to ~10 unique, non-redundant triples.

```
# Add after sorting, before returning results:
seen_keys = set()
deduped   = []
for score, row in scored[:limit]:
    key = (row['source'], row['relation'], row['target'])
    if key not in seen_keys:
        seen_keys.add(key)
        deduped.append(row)
return deduped
```

5.3 Fix 3 — Stopword filtering in question tokeniser

File: test_rag.py **|** **Function:** graph_search()

Impact: Removes noise term matching. 'how' no longer inflates scores of irrelevant triples.

```
# Add at top of graph_search():
STOPWORDS = {'how','what','why','when','the','and',
             'for','are','can','you','do','is','to'}

question_terms = [
    x.lower()
    for x in re.findall(r'[A-Za-z0-9\-]+', question)
    if len(x) > 2 and x.lower() not in STOPWORDS
]
```

5.4 Fix 4 — Module-level resource loading

File: test_rag.py **|** **Function:** vector_search()

Impact: In batch mode (5 questions), the model loads once instead of 5 times. Saves ~10s per batch run.

```
# Move these 4 lines to module level (outside all functions):
_model   = SentenceTransformer(EMBEDDING_MODEL)
_index   = faiss.read_index(str(OUTPUT_DIR / 'manual.faiss'))
_chunks  = json.loads((OUTPUT_DIR / 'chunk_metadata.json')
                      .read_text(encoding='utf-8'))
_triples = list(csv.DictReader(
    open(OUTPUT_DIR / 'graph_triples.csv', encoding='utf-8')))

# Then update vector_search() and graph_search()
# to use _model, _index, _chunks, _triples.
```

5.5 Fix summary

#	Fix	File	Impact	Effort
1	Case-normalise entities	`manual_rag.py`	Removes 6 duplicate graph rows per query	**1 line**
2	Dedup graph results	`test_rag.py`	20 rows → 10 clean rows printed	**6 lines**
3	Stopword filter	`test_rag.py`	Cleaner term scoring signal	**4 lines**
4	Module-level loading	`test_rag.py`	Batch: model loads once not 5×	**Move 4 lines**

6. Known Limitations

6.1 Co-occurrence is not semantic relation

Two entities appearing in the same 1,200-char chunk does not mean they are semantically related in the inferred direction. Example: 'Print' (ACTION) and 'Wi-Fi Settings' (MENU) co-occur in chunk_90 and produce the spurious triple Print --CONFIGURED_VIA--> Wi-Fi Settings.

Mitigation: restrict entity pairing to sentence-level windows using nltk.sent_tokenize(). Entities within the same sentence are more likely to share a genuine relationship.

6.2 RELATED_TO is the majority relation

The infer_relation() function defines 6 specific rules. All other type combinations fall into RELATED_TO. In practice this means most triples carry low semantic precision. Expanding the rule set to cover the most common type pair combinations would significantly improve graph quality.

6.3 No graph-to-vector linkage in chunk_metadata

chunk_metadata.json currently stores only chunk_id, page, and text. It does not store which entities were found in each chunk. Adding an entities[] field at build time would allow the query pipeline to directly link FAISS results to graph nodes without re-running NER.

6.4 Silent page truncation

If the document has more than 100 pages, load_pages() truncates silently. A warning log should be added:

```
if len(all_pages) > MAX_PAGES:
    print(f'WARNING: truncated {len(all_pages)} pages to {MAX_PAGES}')
```

6.5 IndexFlatIP is exact but unscalable

FAISS IndexFlatIP performs brute-force inner product search at O(n·d) per query. This is appropriate for the current corpus size (~140 chunks). For corpora exceeding ~100,000 chunks, an approximate index such as IndexIVFFlat or IndexHNSWFlat should be used.

7. Configuration Reference

Parameter	Value	Location	Effect
MAX_PAGES	100	manual_rag.py	Hard cap on pages loaded from source JSON
CHUNK_SIZE	1200	manual_rag.py	Maximum characters per chunk
CHUNK_OVERLAP	150	manual_rag.py	Characters repeated between adjacent chunks
EMBEDDING_MODEL	all-MiniLM-L6-v2	both scripts	Sentence transformer used at build and query time — must match
top_k	5	test_rag.py	Number of vector chunks returned per query
limit	20	test_rag.py	Maximum graph triples returned per query
vector chunk bonus	+5	test_rag.py	Score added when a triple's chunk_id matches a vector result
term hit bonus	+1	test_rag.py	Score added per question term found in triple blob

8. Glossary

Term	Definition
GraphRAG	Graph-augmented Retrieval-Augmented Generation. Combines knowledge-graph traversal with dense vector retrieval for richer context.
Chunk	A contiguous substring of a page's text, produced by the sliding-window splitter. Each chunk has a unique chunk_id.
Embedding	A fixed-length numerical vector representing the semantic content of a text string. Produced by all-MiniLM-L6-v2.
FAISS	Facebook AI Similarity Search. A library for efficient dense vector similarity search. IndexFlatIP = exact inner product search.
Co-occurrence	Two entities are said to co-occur when they both appear in the same text chunk.
Triple	A (subject, predicate, object) fact in the knowledge graph, e.g. Wi-Fi --CONFIGURED_VIA--> Machine Status.
L2 normalisation	Scaling a vector to unit length. Inner product of two L2-normalised vectors equals their cosine similarity.
IndexFlatIP	FAISS exact brute-force index using inner product as the similarity metric. O(n·d) per query.
hit_chunks	The set of chunk_ids returned by vector search. Used in graph search to give a +5 alignment bonus.
RELATED_TO	Fallback relation type assigned when no specific infer_relation rule matches the source and target entity types.